

Contents

At a glance

Usage

Options

Examples

How each agent is invoked

Preflight checks

The pipeline

1. Issue selection

2. Research

3. Implement → review → repeat

4. Finalization

Commit messages

Artifacts

Operational notes

auto-develop.sh

Autonomous issue-to-merge pipeline. The script picks an open GitHub issue, drives a chain of AI agents through research, implementation, two independent reviews and finalization, and does not move on until the change is pushed and the issue is closed.

It is a driver, not an assistant: once started it keeps consuming issues until it runs out of them or hits `--max-issues`.

At a glance

Property	Value
Location	<code>auto-develop.sh</code> (repository root)
Agents supported	<code>claude</code> , <code>codex</code> , <code>opencode</code>
Required tooling	<code>git</code> , <code>gh</code> (authenticated), <code>jq</code> , plus every agent CLI you select
Package-manager assumption	<code>pnpm</code> — the script runs <code>pnpm lint</code> / <code>pnpm typecheck</code> directly
Works on	the branch you are currently on; it commits and pushes there
Artifacts	<code>logs/issues/<issue-number>/</code>

Usage

```
./auto-develop.sh --research <agent> \  
                  --implement <agent> \  
                  --review-a <agent> \  
                  --review-b <agent> \  
                  [--finalize <agent>] \  
                  [--max-issues <n>] \  
                  [--issue <number>]
```

The four role flags `--research`, `--implement`, `--review-a` and `--review-b` are mandatory; omitting any of them prints the usage text and exits with status 1.

Options

Flag	Meaning
<code>--research <agent></code>	Agent that investigates the issue before any code is written
<code>--implement <agent></code>	Agent that writes the code and tests
<code>--review-a <agent></code>	Reviewer A — judges correctness of the uncommitted changes
<code>--review-b <agent></code>	Reviewer B — judges whether the changes actually solve the issue

Flag	Meaning
<code>--finalize <agent></code>	Agent that commits, pushes and closes. Defaults to the <code>--implement</code> agent
<code>--max-issues <n></code>	Stop after completing <i>n</i> issues. Must be a positive integer
<code>--issue <number></code>	Work on exactly this open issue. Implies <code>--max-issues 1</code> (a larger value is reduced, with a warning)
<code>-h, --help</code>	Print usage and exit

Examples

```
# One issue, opencode for every role
./auto-develop.sh --research opencode --implement opencode \
    --review-a opencode --review-b opencode \
    --finalize opencode --max-issues 1

# Target a specific issue with Claude
./auto-develop.sh --research claude --implement claude \
    --review-a claude --review-b claude --issue 42

# Mix agents: research with Claude, implement with Codex, cross-review
./auto-develop.sh --research claude --implement codex \
    --review-a claude --review-b opencode --max-issues 3
```

How each agent is invoked

Agent	Command
claude	<code>claude --dangerously-skip-permissions -p "<prompt>"</code>
codex	<code>codex exec --yolo "<prompt>"</code>
opencode	<code>opencode run "<prompt>"</code>

All three run with permission prompts disabled. The agents have unattended read/write access to the working tree and may execute commands. Run this only on a repository and branch where that is acceptable.

Preflight checks

Before the first issue is touched the script verifies, in order:

1. Every selected agent name is one of `claude`, `codex`, `opencode`.
2. `git`, `gh` and `jq` are on `PATH`, plus the CLI of each agent actually selected.
3. The current directory is inside a git repository (it then `cd`s to the repo root).
4. `gh auth status` succeeds — otherwise it aborts with "Run 'gh auth login'".
5. `HEAD` is not detached.
6. **The repository owner is `TheMorpheus407`**. This is hard-coded; on any other owner the script aborts. It is a deliberate safety catch against pointing the pipeline at a foreign repository, and it is the first thing to change if the script is ever reused elsewhere.

A dirty working tree is *not* an error — it only produces a warning, and the run continues with the uncommitted changes in place.

The pipeline

1. Issue selection

With `--issue`, the given issue is used and must be `OPEN`; otherwise the script exits.

Without it, the script queries GitHub via GraphQL for open issues **created by the repository owner** and takes the **lowest issue number** — so the backlog is worked oldest-number-first. Issues opened by anyone else are never picked up. When the query returns nothing, the script prints "No open issues found." and exits cleanly.

2. Research

The research agent reads the issue and its comments, reads the relevant code, and writes root-cause analysis, impacted modules, constraints, alternatives, risks and test ideas to `research.md`. It also posts the research as a comment on the issue.

3. Implement → review → repeat

This is the core loop, and it has **no iteration limit** — it repeats until both reviewers accept.

- The **implementer** reads the issue and everything in the issue directory, writes the code and tests, and summarizes to `implementation.md`. It is explicitly told **not to commit**.
- On every repetition after a denial, the previous review files are injected into the prompt as a retry context, so the implementer must read the reviewer feedback before changing anything.
- **Reviewer A** judges correctness of the uncommitted diff: regressions, edge cases, naming, DRY, test quality, security, efficiency. It must run `pnpm lint` and `pnpm typecheck`; a failure of either forces a denial.
- **Reviewer B** judges the same diff against the issue requirements: do the changes actually solve it, completely.
- Both reviewers must put `ACCEPTED` or `DENIED` as the very first word of their review file, and are instructed to deny if *any* concern exists.

The loop exits only when **both** reviews say `ACCEPTED` **and** a final `pnpm lint` run by the script itself passes. Anything else — a denial, a missing review file, an agent that crashed — sends it back to the implementer.

How a verdict is read

`extract_verdict` tries three strategies in order, and the first hit wins:

1. the first word of line 1,
2. the last word of the file,
3. the first standalone `ACCEPTED` or `DENIED` anywhere in the file (the fallback for agents that wrap the verdict in markdown).

An empty or missing file yields no verdict, which counts as "not accepted".

4. Finalization

The script composes the commit message itself (see below) and writes it to `commit-message.txt`. If the working tree turns out to be clean at this point, there is nothing to finalize and the issue is skipped with a warning.

The finalizer agent then stages the changes, runs `pnpm typecheck` and `pnpm lint` until they pass, commits **using exactly that message file**, pushes to the current branch, and closes the issue with a comment.

The script does not take the agent's word for it. After each attempt it independently verifies that

- local `HEAD` equals `origin/<branch>` (`git ls-remote`), and
- the issue state is `CLOSED` (`gh issue view`).

If either check fails it simply runs the finalizer again. **This retry loop is also unbounded** — an agent that can never push (no permissions, protected branch, failing hook) will be retried forever.

Commit messages

The message is derived from the issue's labels, not from the agent, so it always satisfies commitlint. Format:

```
type(scope): description  
  
Closes #<issue-number>
```

The description is the issue title with any milestone prefix (`M0-13:`, `M2:`) stripped, whitespace collapsed, lowercased, trailing period removed, and the header truncated to 100 characters.

Label	→ type		Label	→ scope
bug	fix		frontend	web
testing	test		backend	api
documentation	docs		shared	shared
infrastructure, ci	ci		infrastructure	infra
devtools	chore		ci	ci
enhancement, feature	feat		database	db
(none of the above)	feat		testing	e2e
			documentation	docs
			devtools	config
			(none of the above)	(no scope)

Labels are matched in the order listed, so the first match wins — an issue labelled both `bug` and `enhancement` becomes a `fix`.

Note on conventions. `CONTRIBUTING.md` asks for `Issue #42: <description>` on issue-linked commits and Conventional Commits only for general work. This script always produces Conventional Commits with a `Closes #N` footer. The two documents disagree; the script's format is the one commitlint enforces.

Artifacts

Everything an issue produces lands in `logs/issues/<issue-number>/` :

File	Written by	Contents
<code>issue.json</code>	the script	Snapshot of the issue: title, body, author, labels, assignees, comments
<code>research.md</code>	research agent	Investigation and root-cause analysis
<code>implementation.md</code>	implementer	Summary of changes, files touched, tests run
<code>review-1.md</code>	reviewer A	Correctness verdict + findings
<code>review-2.md</code>	reviewer B	Issue-coverage verdict + findings
<code>commit-message.txt</code>	the script	The exact commit message handed to the finalizer
<code>finalization.md</code>	finalizer	Commands run and final git status

The review files double as the loop's memory: their presence is what triggers the retry context on the next implementation round.

Operational notes

It pushes to the branch you are standing on. There is no branch creation. Check out a working branch before starting, or the pipeline will commit straight to whatever is checked out — including `main` .

Two loops are unbounded. Neither the review loop nor the finalizer loop has an attempt cap or a backoff beyond a fixed 2-second pause. A persistently denying reviewer or a finalizer that cannot push will spin indefinitely, burning API budget. Watch the first runs rather than leaving them unattended.

Agent failures are tolerated, not fatal. Every agent invocation that exits non-zero only produces a warning; the pipeline continues. Missing or empty output files likewise warn and continue. The real gate is the verdict check, not the exit codes.

The tooling is pnpm-specific. `pnpm lint` and `pnpm typecheck` are called directly by the script and are hard-wired into the reviewer prompts.